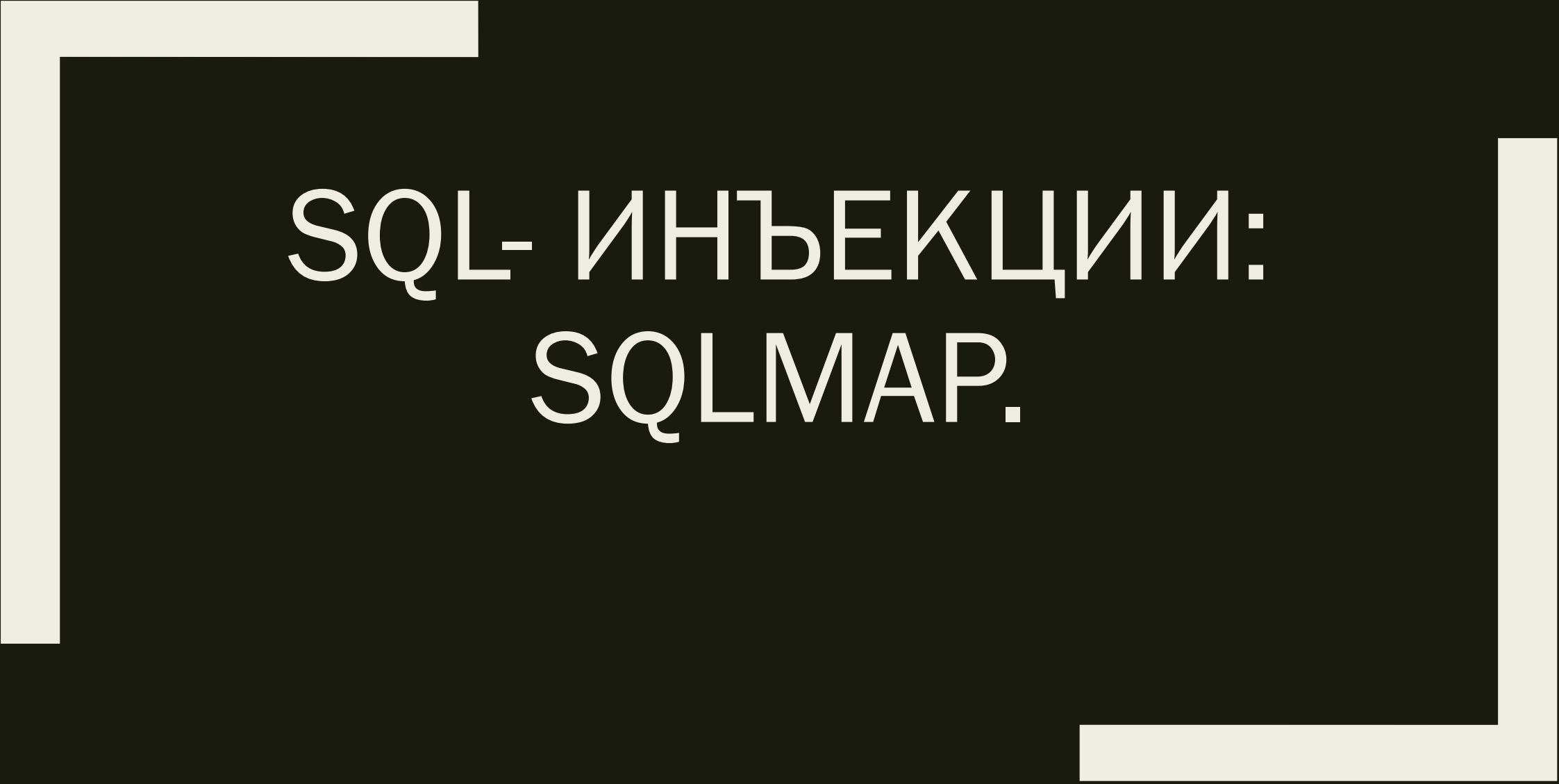


A thick, light-colored L-shaped frame is positioned on the left and bottom edges of the slide, framing the central text.

SQL-ИНЪЕКЦИИ: SQLMAP.

SQLMap

Sqlmap - одна из мощнейших открытых утилит для тестера, которая автоматизирует процесс поиска и эксплуатации SQL-инъекций с целью извлечения данных или захвата удаленного хоста. Что делает sqlmap отличным от других утилит для обнаружения SQL-инъекций, так это возможность эксплуатировать каждую найденную уязвимость. Это означает, что sqlmap способен не только находить "дырку", но еще и использовать ее по полной программе.

SQLMap

А коль уж в качестве задачи ставится именно эксплуатация уязвимости, то сканеру приходится быть особенно внимательным к деталям: он не будет выдавать миллион ложных срабатываний "так, на всякий случай". Любая потенциальная уязвимость дополнительно проверяется на возможность эксплуатации. Сканер идет с огромным функционалом, начиная от возможности определения системы управления базой данных (далее DBMS), создания дампа (копии) данных и заканчивая получением доступа к системе с возможностью обращаться к произвольным файлам на хосте и выполнять на сервере произвольные команды.

SQL-инъекция: что это?

SQL-инъекция — это атака, направленная на веб-приложение, в ходе которой конструируется SQL-выражение из пользовательского ввода путем простой конкатенации (например, `$query="SELECT * FROM users WHERE id=".$_REQUEST["id"]`). В случае успеха атакующий может изменить логику выполнения SQL-запроса так, как это ему нужно. Чаще всего он выполняет простой fingerprinting СУБД, а также извлекает таблицы с наиболее "интересными" именами (например "users"). После этого, в зависимости от привилегий, с которыми запущено уязвимое приложение, он может обратиться к защищенным частям бэк-энда веб-приложения (например, прочитать файлы на стороне хоста или выполнить произвольные команды).

Какие уязвимости может находить SQLMAP?

UNION query SQL injection. Классический вариант внедрения SQL-кода, когда в уязвимый параметр передается выражение, начинающееся с "UNION ALL SELECT". Эта техника работает, когда веб-приложения напрямую возвращают результат вывода команды SELECT на страницу: с использованием цикла for или похожим способом, так что каждая запись полученной из БД выборки последовательно выводится на страницу. **Sqlmap** может также эксплуатировать ситуацию, когда возвращается только первая запись из выборки.

Какие уязвимости может находить SQLMAP?

Error-based SQL injection. В случае этой атаки сканер заменяет или добавляет в уязвимый параметр синтаксически неправильное выражение, после чего парсит HTTP-ответ (заголовки и тело) в поиске ошибок DBMS, в которых содержалась бы заранее известная инъецированная последовательность символов и где-то "рядом" вывод на интересующий нас подзапрос. Эта техника работает только тогда, когда веб-приложение по каким-то причинам (чаще всего в целях отладки) раскрывает ошибки DBMS.

Какие уязвимости может находить SQLMAP?

Stacked queries SQL injection. Сканер проверяет, поддерживает ли веб-приложение последовательные запросы, и, если они выполняются, добавляет в уязвимый параметр HTTP-запроса точку с запятой (;) и следом внедряемый SQL-запрос. Этот прием в основном используется для внедрения SQL-команд, отличных от SELECT, например для манипуляции данными (с помощью INSERT или DELETE). Примечательно, что техника потенциально может привести к возможности чтения/записи из файловой системы, а также выполнению команд в ОС. Правда, в зависимости от используемой в качестве бэк-энда системы управления базами данных, а также пользовательских привилегий.

Какие уязвимости может находить SQLMAP?

Boolean-based blind SQL injection. Реализация так называемой слепой инъекции: данные из БД в "чистом" виде уязвимым веб-приложением нигде не возвращаются. Прием также называется дедуктивным. **Sqlmap добавляет в уязвимый параметр HTTP-запроса синтаксически правильно составленное выражение, содержащее подзапрос SELECT** (или любую другую команду для получения выборки из базы данных). Для каждого полученного HTTP-ответа выполняется сравнение **headers/body** страницы с ответом на изначальный запрос — таким образом, утилита может символ за символом **определить вывод внедренного SQL-выражения**. В качестве альтернативы пользователь может предоставить строку или регулярное выражение для определения "true"-страниц (отсюда и название атаки). Алгоритм бинарного поиска, реализованный в sqlmap для выполнения этой техники, способен извлечь каждый символ вывода максимум семью HTTP-запросами. В том случае, когда вывод состоит не только из обычных символов, сканер подстраивает алгоритм для работы с более широким диапазоном символов (например для unicode'а).

Какие уязвимости может находить SQLMAP?

Time-based blind SQL injection. Полностью слепая инъекция. Точно так же, как и в предыдущем случае, сканер "играет" с уязвимым параметром. Но в этом случае добавляет подзапрос, который приводит к паузе работы DBMS на определенное количество секунд (например, с помощью команд SLEEP() или BENCHMARK()). Используя эту особенность, сканер может посимвольно извлечь данные из БД, сравнивая время ответа на оригинальный запрос и на запрос с внедренным кодом. Здесь также используется алгоритм двоичного поиска. Кроме того, применяется специальный метод для верификации данных, чтобы уменьшить вероятность неправильного извлечения символа из-за нестабильного соединения.

SQLMAP

Несмотря на то что сканер умеет автоматически эксплуатировать найденные уязвимости, нужно детально представлять себе каждую из используемых техник. Важно также понимать, что для разных DBMS реализации атаки зачастую сильно отличаются. Все эти случаи умеет обрабатывать sqlmap и на данный момент поддерживает MySQL, Oracle, PostgreSQL, Microsoft SQL Server, Microsoft Access, SQLite, Firebird, Sybase и SAP MaxDB.

Фишки SQLMAP

Движок для определения SQL-уязвимостей — пускай и самая важная, но все-таки не единственная часть функционала sqlmap. В sqlmap реализовано:

- Извлечение имен пользователей, хешей их паролей, а также привилегий и полей.
- Автоматическое распознавание типа, используемого хеша и возможность взлома его с помощью брутфорса по словарю.
- Получение списка баз данных, таблиц и столбцов.
- Возможность сделать полный или частичный дамп базы данных.
- Продвинутый механизм поиска баз, таблиц или даже столбцов (по всем базам сразу), что может быть полезно для определения таблиц с "интересными" данными вроде имен пользователей (users) или паролей (pass).

Фишки SQLMAP

- Загрузка или, наоборот, закачка произвольных файлов на сервер, если уязвимое веб-приложение использует MySQL, PostgreSQL или Microsoft SQL Server.
- Выполнение произвольных команд и получение шелла, если на хосте используется одна из СУБД, перечисленных в предыдущем пункте.
- Поддержка прямого подключения к базе данных (без явного использования SQL-уязвимости) с использованием полученных в ходе атаки имени и пароля пользователя для доступа к DMBS, а также IP-адреса, порта и имени базы данных.
- Установка надежного TCP-соединения (так называемого out-ofband) между машиной пентестера и хостом, на котором запущен сервер баз данных. В качестве обертки для этого канала может стать интерактивная командная строка (шелл), сессия Meterpreter или доступ к удаленному рабочему столу через VNC-подключение.

Как защититься?

Наиболее надежным способом предотвращения SQL-инъекций является использование параметризованных SQL-параметров. К примеру, в случае с PHP это возможно с помощью пакета PEAR's DB, предлагающего интерфейс для выполнения абсолютно безопасных SQL-выражений. Обращение к БД происходит следующим образом:

```
$p = $db->prepare("SELECT * FROM users WHERE id = ?");  
$db->execute($p, array($_GET['id'])).
```

Как защититься?

Основная идея заключается в том, что если позиция параметров явно задана, то можно абсолютно безопасно передавать SQL-запросы базе данных, исключая возможность для параметров самим стать SQL-выражениями (в том числе зловредными). Стоит заметить, что другие механизмы, такие как использование принудительного приведения типов (например, с помощью функции `intval()`) в связке с экранированием строк такими функциями, как `mysql_real_escape_string()` или `addslashes()`, не являются абсолютно безопасными. Проблема в том, что существуют некоторые варианты для их обхода, а, следовательно, к их использованию необходимо подходить с максимальным вниманием.